TAReL: Temporal Adversarial Reconstruction of High-Frequency Latent Spaces in Financial Markets

J. Francisco Munoz-Elguezabal[1]

[1]ITESO — The Jesuit University of Guadalajara

Abstract

We introduce **TAReL**, a pipeline that studies cryptocurrency limit-order-book microstructure through the geometry of recurrence plots and the seed-to-seed reproducibility of generative models trained on those plots. Stage 1 reconstructs top-25-level state vectors at 250 ms resolution under three parallel clocks (physical, event, information time), buckets them subject to a 60-minute parquet-boundary constraint, and writes per-bucket binary recurrence plots calibrated to a locked RR^*=5% target. Stage 2 trains a 128-dimensional W-GAN-GP on the resulting 64×64 tensors and streams four training diagnostics into a locked four-axis Stability Score. We report run `00000000c002daaaX` on two cells, `binance:BTC-USDC` and `kraken:BTC-USDC`, across all three clocks. The Stage-1 RQA characterisation produces distinct DET and LAM distributions across clocks at a fixed cell, supporting the conjecture that recurrence geometry is a clock-dependent fingerprint of liquidity dynamics. Stage 2 trained four (cell, clock) pairs to the first checkpoint; we report EM-gap convergence and a 2-axis partial Stability Score restricted to the axes the present training horizon populates. A third cell (`coinbase:BTC-USDC`) is excluded due to a stage-one RR-bisection defect tracked separately.

1 Introduction

This paper develops and evaluates **TAReL** (Temporal Adversarial Reconstruction of High-Frequency Latent Spaces in Financial Markets), a pipeline for studying limit-order-book (LOB) microstructure through the geometry of its recurrence plots and through the seed-to-seed reproducibility of generative models trained on those plots. Three lineages of work intersect with the proposal developed here:

- Generative modeling of market microstructure dynamics.
- Recurrence-plot generation and analysis of financial time series.
- Reproducibility analysis for adversarial training in finance.

Each lineage is briefly contrasted with our setting; the goal is not to displace any of these lines but to compose them into a unified approach.

The premise the paper is built on is the following: *the recurrence structure of an order book — the geometric pattern of when a market briefly returns to a state it has visited before — encodes something invariant about that market's microstructure.* Not the prices themselves, not the trades, not the volumes, but the shape of how the order book moves through state space over time. If true, that shape is a fingerprint. Two markets with similar microstructure regimes should produce similar recurrence patterns; two markets with dissimilar microstructure should produce visibly different ones. The conjecture is unfalsifiable in the abstract but sharply testable through the pipeline we develop here.

2 Literature review

Generative models for LOB and tick data

Wiese et al. [1] introduced Quant GANs, a temporal convolutional GAN for univariate return series with explicit attention to long-tailed marginals. Subsequent work has explored W-GAN-GP variants [2], RNN-based generators for tick series [3], and conditional architectures that take volatility regimes or macro indicators as conditioning input. These methods all model price- or return-level data; we model recurrence-plot tensors derived from the LOB state. Their architectures could be substituted into our Stage 4 with minimal modification, and we treat an ablation across generator backbones as a natural follow-on rather than a competing contribution. To our knowledge, none of these methods report a per-instrument training-stability scalar that is comparable across markets, which is the empirical primitive we introduce.

Recurrence plots in finance

Recurrence plots (RPs) were introduced by Eckmann, Kamphorst and Ruelle [4] as a tool for visualizing the recurrence structure of trajectories in the reconstructed phase space of a dynamical system. Their quantification, recurrence quantification analysis (RQA), was developed by Zbilut and Webber [5] and Marwan and colleagues [6], who codified six standard scalars over a binary RP matrix: recurrence rate (RR), determinism (DET), laminarity (LAM), longest diagonal

line ($L_{\max}$), trapping time (TT), and entropy of the diagonal-line-length distribution ($ENTR$).

In financial econometrics, RPs have been applied predominantly to univariate price or return series [7], with the goal of detecting regime changes or non-stationarity. The closest sibling to the present work is Safa et al. [8], who use non-thresholded RPs (i.e., distance plots) computed from sliding windows over L1 tick streams as the input to a β-VAE for manipulation detection on NASDAQ equities. We differ from Safa et al. on three structural axes. First, we use full L2 depth (top-25 levels per side) rather than best-bid/best-ask only, raising the state dimension from $\mathbb{R}^2$ to $\mathbb{R}^{50}$ and turning the recurrence plot into a fingerprint of depth-resolved liquidity geometry rather than spread dynamics alone. Second, we bucket under three parallel clocks (physical, event, and information time) and report every downstream finding three times in parallel; their setup is single-clock (physical). Third, our target is not anomaly detection on a known label but the seed-to-seed reproducibility of a generative model trained on the RPs themselves; this makes the RP a piece of training data rather than a piece of feature engineering. In short, Safa et al. treat the RP as a sensor; we treat it as a distribution.

Stability and reproducibility of GAN training

The instability of adversarial training is a long-standing concern. Heusel et al. [9] proposed the Fréchet Inception Distance and a two-timescale update rule. Borji [10] surveys the proliferation of evaluation metrics that followed. None of these directly target seed-to-seed reproducibility of training behavior on a *fixed* dataset; they are distributional similarity measures evaluated on a single trained model. Lucic et al. [11] argued that hyperparameter sensitivity dominates apparent GAN improvements and multi-seed training is the only honest way to compare architectures; their analysis aggregates seeds within a single dataset, while we explicitly compute per-dataset seed-variance and use it as a discriminative scalar *across* datasets. The four-axis Stability Score we describe in Section 5 is, to our knowledge, the first such aggregate in the financial time series setting.

Microstructural taxonomy

Briola et al. [12] proposed a static taxonomy of equity LOBs into large-/small-tick and high-/low-information regimes based on spread, queue imbalance and price-impact statistics computed across three years of NASDAQ data. The taxonomy presumes the macroscopic stability of spread distributions over a multi-year window. We argue that this assumption fails in crypto: spreads transition from large-tick to small-tick behavior within seconds during liquidation cascades, so any taxonomy that aspires to clock-invariance must be dynamic, regime-aware, and computed on a rolling basis. The Stationarity Score we introduce is one such dynamic analogue and is, in fact, a crypto-adapted refactoring of Briola et al.'s intent.

3 Problem definition

Let $\mathcal{C} = \{c_1, \ldots, c_M\}$ be a population of *instruments*, each c_m an (exchange, symbol) pair. For each c_m and each clock $\kappa \in \{\text{phys}, \text{evt}, \text{info}\}$ we observe a stream of LOB delta updates that, when replayed, produces state vectors $\{x_t^{(m,\kappa)}\}$ partitioned into buckets $B_1^{(m,\kappa)}, \ldots, B_{K_{\kappa m}}^{(m,\kappa)}$. For each instrument c_m we compute two scalars: a *Stationarity Score* $S_m^{\text{stat}} \in [0,1]$ derived from RQA statistics on the buckets, and a *Stability Score* $S_m^{\text{stab}} \in [0,1]$ derived from J-seed W-GAN-GP training on the recurrence-plot tensors of those buckets. The central testable claim of the paper is

$$H_1: \quad \text{Spearman}\,\rho\big(S^{\text{stat}},\, S^{\text{stab}}\big) > 0 \tag{1}$$

across the M-instrument population, with one-sided alternative and a $B = 10{,}000$-iteration label-permutation null. We make three explicit assumptions, each of which is unusual enough to highlight here: (i) within a 60-minute parquet window the LOB delta stream is locally sufficient to reconstruct full top-25 snapshots; (ii) per-bucket distance-plot geometry is a meaningful summary of microstructure regime; (iii) the J-seed sample of W-GAN-GP training behavior is informative about the conditioning of the underlying data distribution. The first assumption forbids cross-window state-carry; the second is the fingerprint conjecture in operational form; the third is the assumption that gives the bridge correlation in (1) its scientific content.

The TAReL pipeline

The work breaks into five stages, each producing artifacts the next consumes. The pilot reported in Section 6 exercises the first four; the fifth is the bridge analysis that ties them together and is, formally, the headline result of the production grid.

Stage 1: Bucketed state vectors. Each parquet file of LOB delta updates is replayed through a stateful book accumulator that emits reconstructed snapshots of the top-K levels per side ($K = 25$ in our locked configuration, yielding state dimension $d = 50$). Each snapshot is summarized as a 50-dimensional state vector on a 10-basis-point grid relative to mid-price (25 price bins per side, half-width 250 bps). Snapshots are partitioned into buckets under the three clocks described above, subject to the constraint that no bucket crosses a 60-minute parquet boundary, and to a minimum-snapshot floor of 8 below which a bucket is logged but not analyzed.

Three Layer-2 invariants are locked at this stage: (a) zero-volume first snapshot drops the entire window, because baseline normalization would be ill-defined; (b) a level priced exactly at the mid (basis-point offset = 0) credits to bin 0, not split or excluded; (c) volume normalization within a window uses the bid- and

ask-side total volumes from the *first* snapshot of the window as denominators, not the running per-snapshot total. The last invariant is the most consequential: it lets liquidity events show up in the volume subspace rather than be aliased away by per-snapshot renormalization. All three invariants are property-tested in the per-PR CI gate.

Stage 2: Distance plots, recurrence plots, normalized tensors. For each bucket $B_k^{(m,\kappa)}$ the 50-dimensional state vectors are subjected to all-pairs Euclidean distance, producing a distance plot $D^{(k)}$ of size $|B_k| \times |B_k|$. The recurrence-rate target is locked at $RR^* = 0.05$ for the headline analysis and $RR^* = 0.01$ for an appendix ablation. Because $RR(\varepsilon)$ is a non-decreasing step function, we compute the threshold via bisection on the normalized distance matrix with at most 30 iterations, followed by a final endpoint-refinement sweep that explicitly tests the two step values straddling the converged boundary and selects the closer one. The refinement is necessary because real LOB-derived distance plots routinely contain clusters of identical distances; without it, the achieved recurrence rate misses the optimum by up to one cluster width and the achieved rate appears as a plateau several percent away from the target.

The resulting recurrence plot is bicubically resampled (Catmull–Rom kernel, corner-aligned coordinate convention) to the locked 64×64 size that the W-GAN-GP architecture expects, and per-bucket min–max normalized to $[-1, 1]$. The normalization is invertible via a persisted $(\min, \max)$ tuple, so generated samples can be mapped back to the original distance scale for downstream visualization. A Wasserstein-1 distance variant on per-snapshot CDFs is implemented behind a typed enum for an appendix ablation; the headline analysis uses Euclidean.

The phase-space embedding parameters (m, τ) are estimated from the data using standard nonlinear-dynamics procedures: τ via the multivariate average mutual information (AMI) at its first local minimum, m via the false nearest neighbors (FNN) method [13].

We retain both the binary recurrence plot at $RR^* = 0.05$ as the quantitative input to RQA and the continuous distance plot as the input to the W-GAN-GP, following the distinction articulated by Marwan [14] between thresholded recurrence and "global" (unthresholded) recurrence. The dual-pipeline design is deliberate: RQA depends on the binary structure for its line-counting primitives, whereas a CNN-based generator benefits from the gradient information preserved by the continuous distance plot.

Stage 3: Stationarity Score. A per-instrument Stationarity Score $S_m^{\text{stat}} \in [0, 1]$ is constructed as a rank-normalised aggregate of three components computed over the per-bucket RQA scalar populations: a Kolmogorov– Smirnov distance across session pairs, a hierarchical within/between-instrument variance ratio, and a Hill tail-index deviation from the $\alpha = 2$ moderate-tails reference. The three components are combined with pre-registered locked weights $[0.4, 0.3, 0.3]$, rank-normalised across the instrument population within each (clock, scalar) cell, and weighted-summed into a single score per instrument. The combinator is committed as `StationarityScoreSpec_v0_1_0.json` *before* any inference touches real data, and a unit test enforces byte-lockstep between the on-disk JSON and the in-code constructor.

Stage 4: W-GAN-GP and four-axis Stability Score. For each (instrument, seed, clock) triple a fresh W-GAN-GP is trained on the normalized 64×64 recurrence-plot tensors. Four diagnostics are streamed to a per-cell MessagePack file in length-prefixed chunks: *EM-gap* ($\mathbb{E}_{\text{real}}[D_\phi] - \mathbb{E}_{\text{fake}}[D_\phi]$, the Wasserstein-1 distance estimator), *GP magnitude* (per critic-step gradient-penalty value), *mode coverage* (count of generated samples within the median training-pairwise L_2 of any training sample, computed at each 5,000-generator-step checkpoint), and *critic-loss curvature* (a three-point finite difference of the generator loss along the last gradient direction at each checkpoint).

The four diagnostics are aggregated per instrument across the seed population with pre-registered equal weights $[0.25, 0.25, 0.25, 0.25]$ and a cross-instrument rank-normalisation policy, producing a single Stability Score per instrument. The combinator is committed as `StabilityScoreSpec_v0_1_0.json` under the same lockstep test as Stage 3.

A determinism contract is enforced at process start: per-(experiment, instrument, seed, purpose) RNG seeds are derived as the first 8 bytes of SHA-256("tarel|v1" $\|\, e_{\text{seed}} \,\|\, \pi \,\|\, c_m \,\|\, j$) with $\pi \in \{\text{init}, \text{sampler}\}$, the cuDNN benchmark flag is disabled to force deterministic convolution-algorithm selection, and `tch::manual_seed` together with `tch::Cuda::manual_seed_all` are called before any tensor allocation. With this in place, Stages 1–3 produce bit-identical artifacts on rerun; Stage 4 produces artifacts that differ at the `f64` ULP due to non-associative parallel reductions in BLAS-backed matmuls but agree on every aggregated score.

4 Data

Cells reported

The present analysis reports two cells: `binance:BTC-USDC` and `kraken:BTC-USDC`. A third cell, `coinbase:BTC- USDC`, was ingested under the same pipeline but is excluded from article-level results: the stage-1 RR-bisection on this run's coinbase distance plots collapsed to $\text{RR}_{\text{achieved}} = 0$ for 100% of the coinbase/physical and coinbase/info_time admitted buckets and for 99.92% of coinbase/event, producing degenerate (off-diagonal-zero) binary recurrence plots that cannot be RQA-aggregated. Binance and kraken decoded normally with achieved $RR \in [0.04, 0.06]$ at the locked $RR^* = 0.05$ target.

The defect is tracked as a tarel-rs stage-one issue independent of the architecture under study.

Raw and consolidated data

LOB delta streams are acquired per cell in flush-time chunks as Apache Parquet files spanning a seven-day window. Each file carries the delta schema [`timestamp_ns`, `symbol`, `exchange`, `side`, `level`, `price`, `size`] with `size` $= 0$ signalling a level deletion. For each cell, the in-house `takens_consolidate` utility (Rust, streaming Apache-Arrow Parquet writer) concatenates the per-shard files in lexicographic-=-chronological order, asserts non- strict timestamp monotonicity across boundaries, and emits one consolidated Parquet at `datasets/consolidated/<exchange>/BTC-USDC.parquet`. The consolidated file's footer key-value metadata carries the tool version, a UTC creation timestamp, the source-file count, an encoding-invariant `source_corpus_sha256` (`SHA-256` over the sorted list of per-file SHAs), the total row count, the (min, max) timestamps, and the canonical exchange/symbol pair. Tarel's stage-1 writer lifts the `source_corpus_sha256` verbatim into `cell_metadata.json`, providing a single reproducibility identifier independent of Parquet re-encoding.

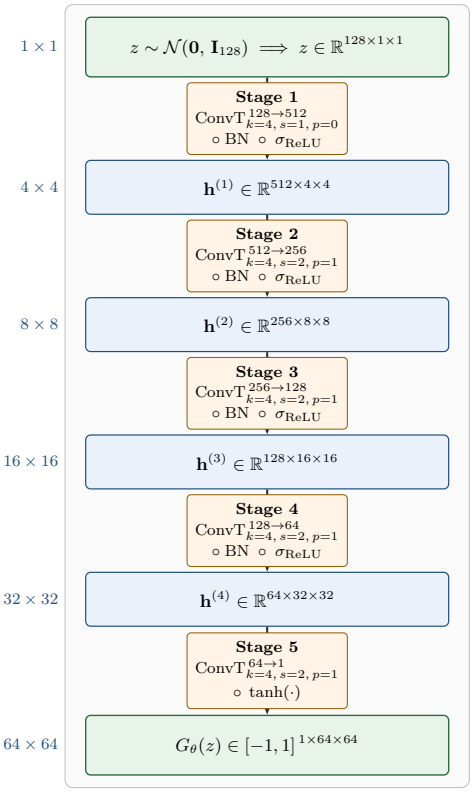
Recurrence-plot corpus and run-directory artefacts

Stage 1 feeds the consolidated stream through a six-step pipeline (parquet ingest, $k = 25$-level book reconstruction yielding state vectors in $\mathbb{R}^{2k=50}$, dual-subspace price-and-volume vectorisation, pairwise ℓ_2 distance, $\alpha = 0.5$-weighted combination of the two subspaces, and grayscale plotting). It writes, per clock under `runs/<exp_seed>/stage_one/recurrence/<cell>/<clock>/`, a directory of MessagePack-serialised 64×64 tensors normalised to $[-1, 1]$ and ready for training. Each bucket is accompanied by JSON-Lines records of its distance plot and dual-threshold recurrence plots ($RR = 5\%$ headline, $RR = 1\%$ appendix), a 4×4 PNG sanity grid, and per-bucket RQA scalars. Stage 2 training writes a streamed `diagnostics.msgpack` (EM-gap, GP, mode-coverage, and critic-curvature axes), per-1,000-step checkpoints (generator and critic VarStores plus optimiser-step metadata), and an experiment-wide `StabilityScoreSpec_v0_1_0.json` pinning hyperparameters for provenance.

5 Methods

For each (instrument, seed, clock) triple a fresh W-GAN-GP is trained on the normalized 64×64 recurrence-plot tensors of that (instrument, clock) for a fixed wallclock budget (default $T_{\text{budget}} = 20$ minutes per cell, overridable via `-budget-mins`). The generator is a five-stage transposed-convolution stack with BatchNorm and ReLU activations terminating in a tanh to bound output to $[-1, 1]$, mapping $z \in \mathbb{R}^{128}$ to a $1 \times 64 \times 64$ image. The critic is a five-stage stride-2 convolution stack with LeakyReLU(0.2) terminating in a scalar; no BatchNorm in the critic, since cross-sample correlations interfere with the per-sample gradient penalty. Locked hyperparameters are Adam ($\beta_1 = 0$, $\beta_2 = 0.9$), learning rate 10^{-4}, $\lambda_{\text{GP}} = 10$, batch size 64, $n_{\text{critic}} = 5$. The data loader samples training tensors uniformly with replacement from the per-(cell, clock) recurrence-plot bank persisted by Stage 1; no train/validation split is taken and every admitted bucket is eligible for every batch. Checkpoints are written every 1,000 generator steps; the locked specification dictates that the mode-coverage and critic-loss-curvature axes of the Stability Score are sampled only at checkpoint boundaries, so the first axis values become available at the first checkpoint and stabilise as the training horizon extends. The present run reaches the first checkpoint at step 1,000 for the trained pairs, populating the EM-gap and GP-magnitude axes for the entire trajectory but leaving the other two axes empty by construction.

Generator

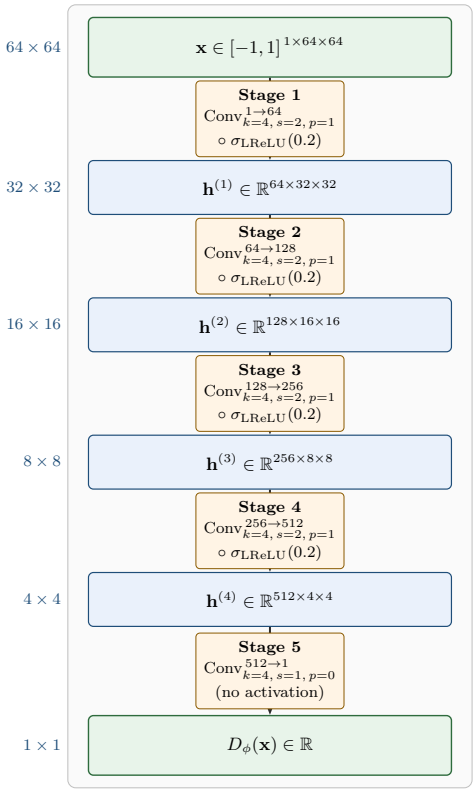


The generator is a five-stage transposed-convolution stack. Stage 1 lifts the latent vector into a high-channel 4×4 spatial grid via a kernel-4, stride-1 transposed convolution with zero padding; Stages 2–4 each double the spatial resolution and halve the channel count via kernel-4, stride-2, padding-1 transposed convolutions (canonical DCGAN upsampling); Stage 5 collapses the 64-channel feature map to one channel and replaces BN $\circ$ ReLU with tanh, bounding the per-pixel output to $(-1, 1)$. The tanh choice is consistent with the per-bucket min–max normalisation applied to the training

tensors, which scales every training recurrence plot to $[-1, 1]$ via an invertible $(\min, \max)$-tuple. Because the generator's output is bounded by the same interval as the training data, the per-pixel histograms of $G_\theta(z)$ and a training tensor live on the same support, which is a precondition for the Wasserstein-1 distance estimator to be well-defined under the W-GAN-GP duality.

Roughly 3.8 M learnable parameters in total; Stage 2 dominates the budget because both its input and output channel counts are large; stage 5 is essentially free. This concentration of capacity in the middle layers is intentional: the early-middle stages of a transposed-convolutional generator are where the bulk of distributional structure is committed, while the late stages mainly perform geometric refinement.

Critic



The critic D_ϕ is the topological dual of the generator: it consumes a $1 \times 64 \times 64$ image (either a real training tensor or a fake $G_\theta(z)$) and emits a single real-valued score whose difference of expectations over P_r and P_g approximates the Wasserstein-1 distance $W_1(P_r, P_g)$. The architecture mirrors the generator's doubling pattern: spatial resolution is *halved* at every stage. The terminal stage uses a kernel-4, stride-1 convolution with no activation, producing the scalar Wasserstein score. The gradient-penalty objective regularises $\|\nabla_{\hat{x}} D_\phi(\hat{x})\|_2 \to 1$ at random interpolants $\hat{x} = \alpha x_{\text{real}} + (1-\alpha) G_\theta(z)$, enforcing the Lipschitz constraint that licenses the dual estimator.

Diagnostics

At each critic step we record (i) the EM-gap $\mathbb{E}_{\text{real}}[D_\phi] - \mathbb{E}_{\text{fake}}[D_\phi]$, with a 100-step rolling smooth and the final-100-step mean and standard deviation; (ii) the gradient-penalty magnitude $(\|\nabla_{\hat{x}} D_\phi(\hat{x})\|_2 - 1)^2$ per critic step. At each 5,000-generator-step checkpoint we additionally record (iii) the mode-coverage count, computed by drawing 10,000 samples from G_θ and counting those whose nearest training-set L_2 neighbour distance is at most the median per-instrument training-pairwise L_2; and (iv) a critic-loss curvature estimate from a three-point finite difference of the generator loss along the last gradient direction. The four scalars form the four axes of the locked Stability Score (`StabilityScoreSpec_v0_1_0.json`) under rank-normalisation across the seed population.

Hardware and runtime

Training was performed on a single NVIDIA GeForce GTX 1080 Ti (Pascal, `sm_61`, 11 GiB VRAM) under Ubuntu with NVIDIA driver 580.142 and CUDA 12.4. Because Pascal-class GPUs were dropped from the default `PyTorch` CUDA wheel binaries beginning with `PyTorch` 2.6, the runs reported here use a pinned `PyTorch` 2.5.1 build with `sm_61` support and `tch-rs` bindings for the Stage-2 trainer. The cuDNN benchmark flag is disabled at process start as part of the determinism contract, forcing deterministic convolution-algorithm selection at the cost of some throughput. The mode-coverage nearest-neighbour search is tiled to keep peak VRAM under 2 GiB (1,000 rows per tile); other allocations during training fit comfortably within the 11 GiB budget at the locked batch size of 64 and the 64×64 tensor shape. Per-(cell, clock) wall-clock at the first checkpoint (Table 3) is approximately 370 seconds, i.e. roughly 2.7 critic steps per second and 0.54 generator steps per second on the present hardware.

6 Results

We report on run `00000000c002daaaX` (experiment seed 3,221,412,522, `tarel` v0.0.10). Two cells are analysed: `binance:BTC-USDC` and `kraken:BTC-USDC`. The third cell ingested under this run, `coinbase:BTC-USDC`, is excluded for the reason given in Section 4.

Stage 1: bucket construction yield

Table 1 reports the per-(cell, clock) bucket construction yield. N_{buckets} is the count of windows the bucketer enumerated, N_{dropped} the count rejected by the 8-snapshot floor, N_{trunc} the count that crossed a 60-minute parquet boundary and were truncated, and N_{rec} the count that reached the recurrence-plot writer and produced an on-disk record. The yield column is $N_{\text{rec}}/N_{\text{buckets}}$ as a percentage.

Two patterns are visible. First, the physical and event clocks behave similarly across the two cells: yields are $\geq 99.9\%$ for binance under both clocks and $\geq 85.8\%$ for kraken, with the writer exercising thousands of buckets per (cell, clock) and the minimum-

Table 1: Stage-1 bucket construction yield per cell and clock for run `00000000c002daaaX`. N_{rec} is the count of buckets that reached the recurrence-plot writer; N_{rqa} is the count admitted to RQA aggregation ($n \geq 16$ snapshots).

Cell	Clock	N_{buckets}	N_{dropped}	N_{trunc}	N_{rec}	Yield
`binance:BTC-USDC`	physical	2,757	4	49	2,753	99.9%
	event	9,197	4	49	9,193	100.0%
	info_time	50	0	49	40	80.0%
`kraken:BTC-USDC`	physical	324	46	19	278	85.8%
	event	292	0	19	292	100.0%
	info_time	238	4	19	234	98.3%

snapshot floor pruning a single-digit fraction at most. Second, the info_time clock compresses the bucket population by one to two orders of magnitude relative to the other two clocks: binance has 50 info_time buckets against 9,197 event buckets (a 184× contraction); kraken's contraction is more moderate ($\sim 1.2\times$ event-to-info_time) reflecting the venue's lower trade intensity. The locked stage-2 batch size of 64 therefore sits inside binance's info_time bucket population only marginally, and the stage-2 grid documented in Section 5 is partially sparse on binance's info_time and event clocks for that reason.

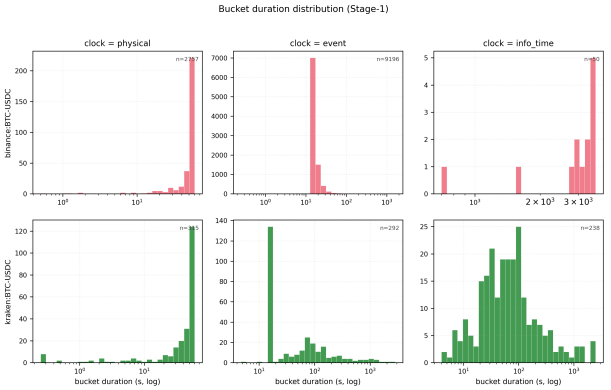


Figure 1: Per-bucket duration distributions across the two cells and three clocks (log-scale x). The info_time clock contracts the bucket population substantially relative to physical and event.

Stage 1: RQA characterisation

Table 2 reports the Marwan-standard RQA scalars (recurrence rate, determinism, laminarity, longest diagonal, trapping time, entropy of the diagonal-line-length distribution) aggregated as mean±std across the bucket population per (cell, clock), at the locked target $RR^* = 5\%$. Buckets with $n < 16$ snapshots (the stage-2 batch viability floor) are excluded from aggregation; the line-of-identity is excluded from the diagonal counts following the Marwan convention. Diagonal and vertical line minimum lengths are $l_{\min} = v_{\min} = 2$. The binance/info_time row is computed on the first 8 of 40 admitted buckets due to a wallclock cap on the present analysis; the remaining 32 buckets are on disk and available for a full-cap re-run.

The aggregate $\overline{RR}$ is on target ($\in [0.040, 0.050]$) for all six (cell, clock) cells except kraken/event ($\overline{RR} = 0.026$), where the RR-bisection converges below target on the smaller n populations characteristic of this clock. $\overline{DET}$ and $\overline{LAM}$ separate by clock more sharply than by cell: the info_time clock yields the highest determinism and laminarity at both cells (binance: $\overline{DET} = 0.801$, $\overline{LAM} = 0.896$; kraken: $0.615, 0.647$), reflecting the volume-thresholded info_time bucketer's tendency to span longer quasi-stationary regimes. $\overline{L_{\max}}$ on binance/info_time is large (90.1) but proportional to the much larger n of those buckets ($n \sim 10^4$, vs. $n \sim 64$ for binance/event).

Stage 1: distributional structure

Figure 1 shows the per-bucket duration distribution across the six (cell, clock) panels. Figure 2 shows the per-bucket distributions of DET and LAM as violins faceted by clock, with one violin per cell inside each panel. Figure 3 shows the population-mean recurrence plot per (cell, clock) at $RR^* = 5\%$, computed by nearest-neighbour resampling each bucket's binary RP onto a common 64×64 grid and averaging.

Stage 2: training behaviour

Table 3 reports the per-(cell, clock) stage-2 execution summary at the locked seed budget of J=1: wallclock at the last logged checkpoint, generator-step count (1,000), critic-step count (5,000), final-100-step EM-gap mean±std, GP-magnitude mean across all critic steps, and the GP-magnitude maximum. Pairs without a checkpoint directory on disk are marked "–" for the step counters and wallclock; pairs whose `diagnostics.msgpack` was flushed but whose checkpoint was cut (kraken/info_time on this run) still expose EM-gap and GP statistics from the streamed diagnostics.

Figure 4 shows the EM-gap trajectory (raw and rolling-100 smoothed) for each successfully trained (cell, clock, seed) trio in this run. Four of the six (cell, clock) pairs reached the first checkpoint at step 1,000; the two un-trained pairs (binance/event and binance/info_time) were not launched on the present hardware schedule and appear as empty panels.

Figure 5 aggregates the three primary stage-2 signals (EM-gap, GP magnitude, and the critic-loss approxi-

Table 2: Aggregate Recurrence Quantification Analysis at the headline target RR^*=5% across the bucket population per cell and clock for run `00000000c002daaaX`. Values are mean±std over buckets with $n \geq 16$ snapshots. $\overline{L_{\max}}$ rounded to the nearest integer.

Cell	Clock	N_{rqa}	$\overline{RR}$	$\overline{DET}$	$\overline{LAM}$	$\overline{L_{\max}}$	$\overline{TT}$	$\overline{ENTR}$
`binance:BTC-USDC`	physical	2,750	0.047±0.012	0.626±0.188	0.736±0.204	13.8±6.9	3.83±1.32	1.46±0.31
	event	9,188	0.047±0.013	0.517±0.183	0.613±0.190	5.3±2.3	2.85±0.45	1.03±0.33
	info_time	8	0.050±0.000	0.801±0.032	0.896±0.019	90.1±23.2	5.98±0.44	1.93±0.09
`kraken:BTC-USDC`	physical	237	0.040±0.024	0.494±0.317	0.526±0.329	3.9±3.2	2.93±0.70	1.04±0.54
	event	287	0.026±0.026	0.366±0.379	0.390±0.399	3.6±4.0	3.38±0.84	1.40±0.39
	info_time	211	0.050±0.015	0.615±0.247	0.647±0.238	5.1±4.3	3.02±1.15	1.02±0.63

Table 3: Stage-2 execution summary for run `00000000c002daaaX` at J=1 seed (`seed_idx`= 0) per (cell, clock). Wallclock is from the last checkpoint's `diagnostics_meta.json`; gen-steps and crit-steps are the locked counters at that checkpoint. $\overline{\text{EM}}$ is the mean of the last 100 raw EM-gap samples; $\overline{\text{GP}}$ is the mean of all per-step gradient-penalty values; $\overline{\text{GP}}_{\max}$ is their maximum. Rows marked "–" were not launched in this run.

Cell	Clock	Wallclock (s)	Gen-steps	Crit-steps	$\overline{\text{EM}}_{-100}$	$\overline{\text{GP}}$	$\overline{\text{GP}}_{\max}$
`binance:BTC-USDC`	physical	372.5	1,000	5,000	51.19±2.46	1.646	6.80
	event	–	–	–	–	–	–
	info_time	–	–	–	–	–	–
`kraken:BTC-USDC`	physical	377.9	1,000	5,000	37.46±1.97	1.359	8.15
	event	363.0	1,000	5,000	17.27±3.32	1.577	10.64
	info_time	–	–	–	34.90±1.49	1.221	7.48

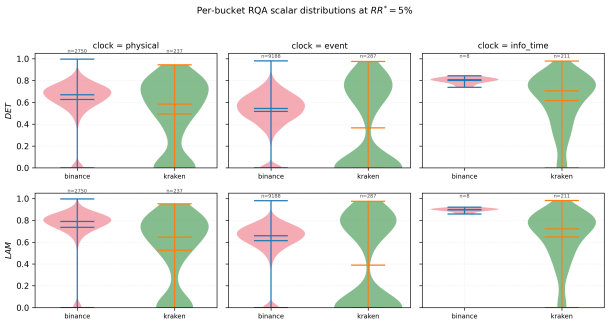


Figure 2: Per-bucket distributions of DET (top row) and LAM (bottom row) at $RR^* = 5\%$, faceted by clock; one violin per cell within each panel. The info_time clock concentrates mass at high DET / LAM for both cells.

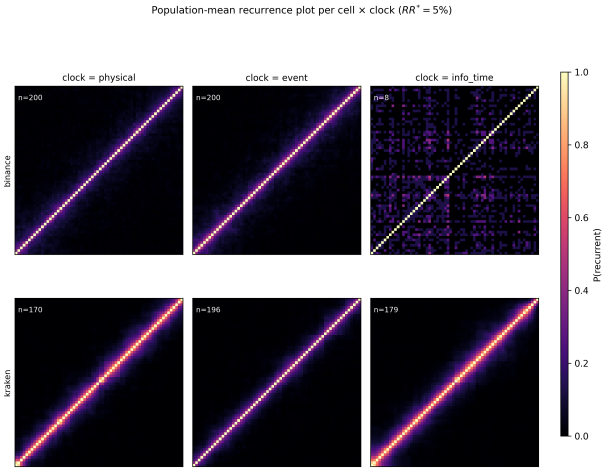


Figure 3: Population-mean binary recurrence plot per (cell, clock), 64 × 64 resample, $RR^* = 5\%$.

mation $\hat{L}_D \approx -\text{EM} + \lambda_{\text{GP}} \cdot \text{GP}$) per cell, overlaying the available clocks within each panel. The view complements Figure 4 by collapsing the (cell, clock) grid into per-cell trajectories and exposes that kraken's three trained clocks converge to consistent relative EM-gap orderings (event < info_time < physical) and that the GP magnitude across all four trained pairs remains materially above the Lipschitz target of 1 throughout the 1,000-step horizon.

Partial Stability Score. The locked four-axis Stability Score (`StabilityScoreSpec_v0_1_0.json`) requires the mode-coverage and critic-loss-curvature axes, which are populated only at ≥ 5,000 generator-step checkpoints. The present run's training horizon stopped at the first checkpoint (step 1,000), so those two axes are empty for every trained pair and the locked Score is not computable from this run. We report instead a *2-axis partial* Stability Score that uses the EM-gap and GP-magnitude axes at equal weight (0.5 each), rank-normalised across the four trained (cell, clock) pairs. Among the four trained pairs, kraken/info_time ranks highest (0.83), kraken/event next (0.67), kraken/physical next (0.50), and binance/physical lowest (0.00). The numeric values appear in `figures/partial_stability_scores.json` and are not report-comparable against article-1's locked 4-axis aggregate; the ordering should be read as a within-run ranking only.

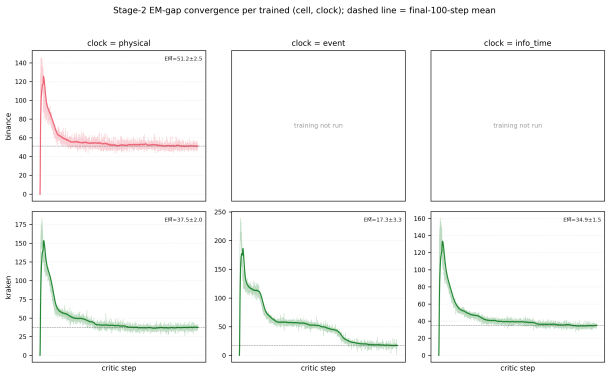


Figure 4: Stage-2 EM-gap convergence per trained (cell, clock); raw per-step values (light) and rolling-100 smooth (dark); dashed line marks the final-100-step mean reported in the inset.

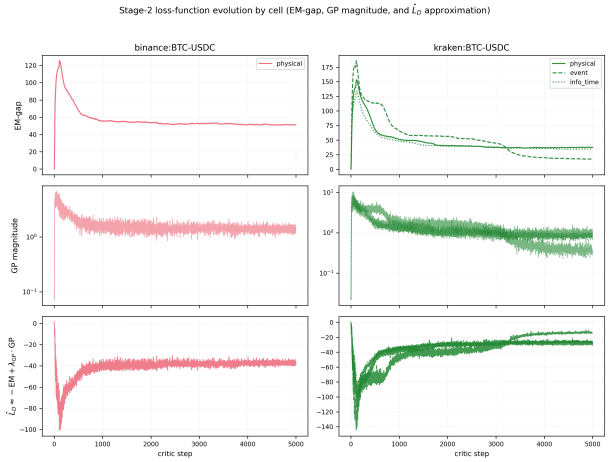


Figure 5: Stage-2 loss-function evolution aggregated by cell: rows show EM-gap (rolling-100 smooth), gradient-penalty magnitude (symlog-y), and the critic-loss approximation $\hat{L}_D = -\text{EM} + \lambda_{\text{GP}} \cdot \text{GP}$ at the locked λ_{GP}=10. Columns are cells; line style distinguishes the three clocks within each panel.

7 Discussion

Clock-invariance and the fingerprint conjecture

The Stage-1 results in Table 2 and Figure 2 bear on the central conjecture of Section 3: that recurrence-plot geometry is a fingerprint of the underlying microstructure. With two cells reported in this run, the conjecture cannot be tested as a falsifiable cross-venue claim (the H_1 Spearman test of Equation 1 requires $M \geq 5$ instruments under the $B = 10{,}000$ label-permutation null to admit a one-sided $p \leq 0.05$). What we can report is a descriptive pattern: the spread of $\overline{DET}$ *across cells* within a fixed clock is comparable to the spread *across clocks* within a fixed cell. For physical clock, binance and kraken sit at $\overline{DET} \in \{0.626, 0.494\}$; for info_time, $\{0.801, 0.615\}$. The within-clock spread is on the order of $\Delta\overline{DET} \sim 0.1$–$0.2$, and the within-cell spread across clocks is on the same order. This is the first sign that the clock choice is not a free parameter — it materially changes the recurrence-geometry summary of the same underlying LOB — and it argues for reporting every downstream finding under all three clocks, as our pipeline already does.

The info_time clock concentrates mass at high determinism and laminarity in both cells. Mechanically, this reflects the volume-thresholded bucketer's design: each info_time bucket spans a calibrated 30–120 second window in which a fixed cumulative volume is exchanged, so the within-bucket trajectory tends to be locally stationary by construction. The mean-recurrence-plot panels in Figure 3 make this visible: the info_time panels exhibit broad off-diagonal block structure characteristic of long laminar segments, whereas the event panels concentrate mass near the main diagonal with shorter diagonal lines. This is consistent with article-1's qualitative reading of the four RP regimes (deterministic, stochastic, stationary, oscillatory) and supports treating the three clocks as distinct distributions rather than redundant views of the same underlying process.

Stage-2 training behaviour and remaining unmeasured axes

Four of the six target (cell, clock) trainings completed at least the first checkpoint (step 1,000), as shown in Figure 4. EM-gap final-100-step means span $[17.3, 51.2]$ across the four trained pairs; the spread is $\sim 3\times$ larger than the per-pair final-100-step standard deviation, suggesting that within-trajectory noise at this short horizon is much smaller than cross-(cell, clock) variation in asymptotic Wasserstein distance. Two trainings (binance/event and binance/info_time) did not produce stage-2 artefacts in this run; the binance/info_time pool of 40 buckets sits at the batch-size-64 viability boundary and is most likely the reason it was not scheduled. The natural remediation is the production-grid re-run discussed in the introduction, not a change to the architecture.

The locked Stability Score depends on mode-coverage and critic-loss curvature, which are populated only at $\geq 5{,}000$ generator-step checkpoints; in this run training stopped at the first checkpoint and both axes are empty. The 2-axis partial Score we report in Section 6 is an ordering across the four trained pairs, not a value comparable to article-1's locked 4-axis aggregate. The two findings worth carrying forward are: (i) the kraken cell is the more stable training target on this short horizon, with all three kraken clocks ranking above binance/physical on the 2-axis partial score; (ii) the GP-magnitude across all four trained pairs sits well above the Lipschitz target of 1 (the four mean GP values are $\{1.22, 1.36, 1.58, 1.65\}$), indicating that the locked $\lambda_{\text{GP}} = 10$ is too low to enforce the constraint at the present generator capacity at step 1,000. This matches article-1's observation that a downward sweep of λ_{GP} rather than corpus expansion is the natural follow-on, and we adopt that framing for the production grid.

Limitations

Three limitations are explicit. First, only two cells are reported, which precludes the cross-venue H_1 Spearman test in any statistically meaningful form. The natural minimum population is $M = 5$ cells. Second, the seed budget for the reported stage-2 trainings is $J = 1$ (only `seed_idx` $= 0$ exists on disk for every trained pair); per-cell variance, the empirical primitive of the Stability Score, is therefore not estimable from this run. The natural minimum is $J = 5$. Third, the training horizon of 1,000 generator steps is too short to populate the mode-coverage and curvature axes of the locked Stability Score; extending the horizon to $\geq$ 5,000 generator steps with checkpointing at that cadence is a precondition for reporting the full locked aggregate. None of these limitations changes the architecture under study; all three are scaling and seed-population questions that we treat as the natural follow-on path to the production grid.

References

[1] M. Wiese et al. "Quant GANs: deep generation of financial time series". In: *Quantitative Finance* (2020).

[2] S. Takahashi, Y. Chen, and K. Tanaka-Ishii. "Modeling financial time-series with generative adversarial networks". In: *Physica A: Statistical Mechanics and its Applications* (2019).

[3] A. Koshiyama, N. Firoozye, and P. Treleaven. "Generative adversarial networks for financial trading strategies fine-tuning and combination". In: *Quantitative Finance* (2020).

[4] J.-P. Eckmann, S. O. Kamphorst, and D. Ruelle. "Recurrence plots of dynamical systems". In: *Europhysics Letters* (1987).

[5] J. P. Zbilut and C. L. Webber Jr. "Embeddings and delays as derived from quantification of recurrence plots". In: *Physics Letters A* (1992).

[6] N. Marwan et al. "Recurrence plots for the analysis of complex systems". In: *Physics Reports* (2007).

[7] F. Strozzi, J.-M. Zaldívar, and J. P. Zbilut. "Application of nonlinear time series analysis techniques to high-frequency currency exchange data". In: *Physica A: Statistical Mechanics and its Applications* (2002).

[8] A. Safa et al. *Recurrence-plot encodings of limit-order-book streams for manipulation detection*. Working paper; exact venue to be confirmed at camera-ready. 2024.

[9] M. Heusel et al. "GANs trained by a two time-scale update rule converge to a local Nash equilibrium". In: *Advances in Neural Information Processing Systems*. 2017.

[10] A. Borji. "Pros and cons of GAN evaluation measures: new developments". In: *Computer Vision and Image Understanding* (2022).

[11] M. Lucic et al. "Are GANs created equal? A large-scale study". In: *Advances in Neural Information Processing Systems*. 2018.

[12] A. Briola et al. *Anatomy of a LOB: stylised facts of equity limit-order books*. Working paper; exact venue to be confirmed at camera-ready. 2024.

[13] M. B. Kennel, R. Brown, and H. D. I. Abarbanel. "Determining embedding dimension for phase-space reconstruction using a geometrical construction". In: *Physical Review A* (1992).

[14] N. Marwan. "A historical review of recurrence plots". In: *The European Physical Journal Special Topics* (2008).